Clean Code - System Design Notes

Single Responsibility Principle (SRP)

A class should have one reason to change.

Bad Example

```
class UserManager:
    def login(self): pass
    def register(self): pass
    def send_email(self): pass
    def generate_report(self): pass
```

Good Example

```
class AuthService:
    def login(self): pass
    def register(self): pass

class EmailService:
    def send_email(self): pass

class ReportService:
    def generate_report(self): pass
```

Separation of Concerns

Bad Example

```
class App:
    def run(self):
        model = YOLOModel(device="gpu")
        result = model.detect("frame")
        save_to_db(result)
```

Good Example

```
class App:
    def __init__(self, model, repository):
        self.model = model
        self.repo = repository

    def run(self):
        result = self.model.detect("frame")
        self.repo.save(result)
```

Factory Pattern

Bad Example

```
model = YOLOModel(device="gpu")
```

Good Example

```
class ModelFactory:
    def create(self, device):
        return YOLOModel(device=device)

model = ModelFactory().create("gpu")
```

Event-Driven Architecture

Bad Example

```
class Feature:
    def process(self):
        db.save(result)
```

Good Example

```
class Feature:
    def __init__(self, event_bus):
        self.event_bus = event_bus

    def process(self):
        self.event_bus.publish("RESULT", result)
```

Centralized Logging

Bad Example

```
print("start")
print("end")
```

Good Example

```
class Logger:
    def log(self, msg):
        print(msg)
```

Domain-Specific Language (DSL)

Bad Example

```
if ear < 0.2 and time > 2:
    alert()
```

Good Example

```
if is_drowsy(face):
    alert_drowsiness()
```

Key Quotes

“Clean code is not written by following rules, but by programmers who care.” “Don’t design for the future, design for change.” “Classes should be small. Smaller than that.” “Systems must be clean too.”